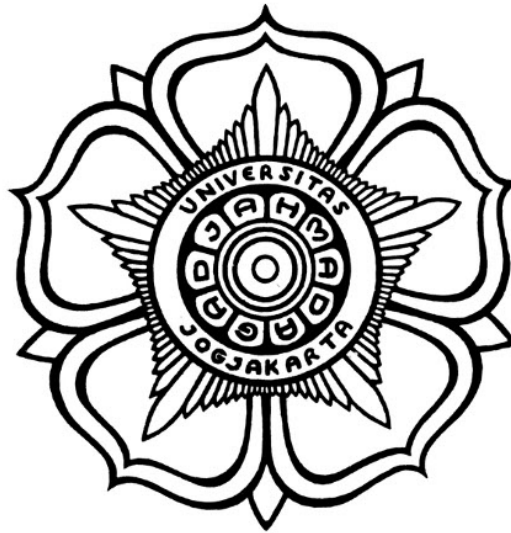


«JUDUL SKRIPSI»

SKRIPSI



THE SUSTAINABLE DEVELOPMENT GOALS
Industry, Innovation and Infrastructure
Affordable and Clean Energy
Climate Action

Disusun oleh:

«AUTHOR»

«NIM»

PROGRAM SARJANA PROGRAM STUDI «NAMA PRODI»
DEPARTEMEN TEKNIK ELEKTRO DAN TEKNOLOGI INFORMASI
FAKULTAS TEKNIK UNIVERSITAS GADJAH MADA
YOGYAKARTA
«TAHUN PENDADARAN»

HALAMAN PENGESAHAN

«JUDUL SKRIPSI»

SKRIPSI

Diajukan Sebagai Salah Satu Syarat untuk Memperoleh
Gelar Sarjana Teknik
pada Departemen Teknik Elektro dan Teknologi Informasi
Fakultas Teknik
Universitas Gadjah Mada

Disusun oleh:

«AUTHOR»

«NIM»

Telah disetujui dan disahkan

Pada tanggal

Dosen Pembimbing I

Dosen Pembimbing II

Dosen Pembimbing 1, S.T., M.Eng., PhD.

«NIP xxxxxx»

Dosen Pembimbing 2, S.T., M.Eng., PhD.

«NIP xxxxxx»

PERNYATAAN BEBAS PLAGIASI

Saya yang bertanda tangan di bawah ini :

Nama :
NIM :
Tahun terdaftar :
Program : Sarjana
Program Studi :
Fakultas : Teknik Universitas Gadjah Mada

Menyatakan bahwa dalam dokumen ilmiah Skripsi ini tidak terdapat bagian dari karya ilmiah lain yang telah diajukan untuk memperoleh gelar akademik di suatu lembaga Pendidikan Tinggi, dan juga tidak terdapat karya atau pendapat yang pernah ditulis atau diterbitkan oleh orang/lembaga lain, kecuali yang secara tertulis disitasi dalam dokumen ini dan disebutkan sumbernya secara lengkap dalam daftar pustaka.

Dengan demikian saya menyatakan bahwa dokumen ilmiah ini bebas dari unsur-unsur plagiasi dan apabila dokumen ilmiah Skripsi ini di kemudian hari terbukti merupakan plagiasi dari hasil karya penulis lain dan/atau dengan sengaja mengajukan karya atau pendapat yang merupakan hasil karya penulis lain, maka penulis bersedia menerima sanksi akademik dan/atau sanksi hukum yang berlaku.

Yogyakarta, tanggal-bulan-tahun

Materai Rp10.000

(Tanda tangan)

Nama Mahasiswa
NIM

HALAMAN PERSEMBAHAN

Tuliskan kepada siapa skripsi ini dipersembahkan!

contoh

KATA PENGANTAR

Contoh: Puji syukur ke hadirat Allah SWT atas limpahan rahmat, karunia, serta petunjuk-Nya sehingga tugas akhir berupa penyusunan skripsi ini telah terselesaikan dengan baik. Dalam hal penyusunan tugas akhir ini penulis telah banyak mendapatkan arahan, bantuan, serta dukungan dari berbagai pihak. Oleh karena itu pada kesempatan ini penulis mengucapkan terima kasih kepada:

1. Orang 1 yang telah
2. Orang 2 yang telah
3. <isi dengan nama orang lainnya>

Akhir kata penulis berharap semoga skripsi ini dapat memberikan manfaat bagi kita semua, aamiin.

Catatan: setiap nama yang dituliskan boleh disertai dengan alasan berterima kasih.

DAFTAR ISI

HALAMAN PENGESAHAN	ii
PERNYATAAN BEBAS PLAGIASI	iii
HALAMAN PERSEMBAHAN	iv
KATA PENGANTAR	v
DAFTAR ISI	vi
DAFTAR TABEL	ix
DAFTAR GAMBAR	x
DAFTAR SINGKATAN.....	xi
INTISARI.....	xii
ABSTRACT	xiii
BAB I Pendahuluan	1
1.1 Latar Belakang	1
1.2 Rumusan Masalah	2
1.3 Tujuan Penelitian	3
1.3.1 Tujuan perancangan dan integrasi perangkat keras	3
1.3.2 Tujuan integrasi SoC dan perangkat lunak.....	3
1.3.3 Tujuan verifikasi dan validasi.....	3
1.4 Batasan Penelitian	3
1.4.1 Batasan perangkat keras dan RTL	3
1.4.2 Batasan SoC dan perangkat lunak.....	4
1.4.3 Batasan pengujian dan dokumentasi	4
1.5 Manfaat Penelitian	4
1.5.1 Manfaat akademik	4
1.5.2 Manfaat praktis	4
1.5.3 Manfaat pengembangan keahlian	4
1.6 Sistematika Penulisan.....	5
BAB II Tinjauan Pustaka dan Dasar Teori	6
2.1 Tinjauan Pustaka	6
2.1.1 Penelitian tentang implementasi I2S pada FPGA	6
2.1.2 Penelitian tentang prosesor soft-core dan integrasi SoC	6
2.1.3 Penelitian tentang AXI4-Lite dan buffering data	7
2.1.4 Gap penelitian dan kontribusi	7
2.2 Dasar Teori	8
2.2.1 Protokol komunikasi I2S	8
2.2.1.1 Sinyal I2S	8
2.2.1.2 Format data Philips I2S.....	8

2.2.1.3	Relasi <i>sample rate</i> , BCLK, dan format slot	8
2.2.2	Audio digital, bit depth, dan DAC PCM5102A	8
2.2.3	FIFO audio dan interupsi watermark	9
2.2.4	Clocking Wizard dan frekuensi aktual	9
2.2.5	MicroBlaze V dan memory-mapped I/O	9
2.2.6	AXI4-Lite	9
2.2.7	Verilog HDL dan perancangan RTL	10
2.2.8	Analisis pemilihan metode	10
BAB III Metode Penelitian.....		11
3.1	Spesifikasi dan arsitektur IP I2S TX	11
3.1.1	Daftar fitur IP	11
3.1.2	Blok diagram perancangan IP	11
3.1.3	Register map	11
3.1.4	Aturan packing data	12
3.1.5	Integrasi Vivado Block Design	12
3.2	Alat dan Bahan	13
3.2.1	Perangkat keras	13
3.2.1.1	Board FPGA	13
3.2.1.2	Modul DAC audio	13
3.2.1.3	Perangkat bantu	13
3.2.2	Perangkat lunak	13
3.3	Metode yang Digunakan	13
3.3.1	Metode perancangan hardware	13
3.3.2	Metode verifikasi dan simulasi	14
3.3.3	Metode integrasi SoC	14
3.3.4	Metode pengembangan software	14
3.3.5	Metode pengujian hardware	15
3.3.6	Pengukuran dan evaluasi	15
3.4	Alur Tugas Akhir	15
BAB IV Hasil dan Pembahasan		17
4.1	Implementasi RTL dan integrasi Vivado	17
4.2	Konfigurasi Clocking Wizard	17
4.3	Simulasi RTL (opsional)	18
4.4	Pengujian dengan ILA	18
4.5	Pengujian audio pada PCM5102A	18
4.6	Evaluasi kinerja	18
4.7	Kendala dan solusi	19
4.8	Pembahasan terhadap tujuan penelitian	20
BAB V Tambahan (Opsional)		21

5.1	Perbandingan dengan referensi desain	21
BAB VI Kesimpulan dan Saran		22
6.1	Kesimpulan	22
6.2	Saran	22
DAFTAR PUSTAKA		23
LAMPIRAN		L-1
L.1	Hasil resource utilization	L-1
L.1.1	Post-implementation resource report	L-1
L.1.2	Timing report	L-1
L.2	Timing diagram	L-1
L.2.1	I2S audio transmission timing	L-1
L.3	Log hasil pengujian	L-2
L.3.1	I2S audio playback test	L-2
L.4	Deskripsi block design	L-2
L.5	Panduan Latex	L-4
L.5.1	Syntax Dasar	L-4
L.5.1.1	Penggunaan Sitasi	L-4
L.5.1.2	Penulisan Gambar	L-4
L.5.1.3	Penulisan Tabel	L-4
L.5.1.4	Penulisan formula	L-5
L.5.1.5	Contoh list	L-5
L.5.2	Blok Beda Halaman	L-5
L.5.2.1	Membuat algoritma terpisah	L-5
L.5.2.2	Membuat tabel terpisah	L-6
L.5.2.3	Menulis formula terpisah halaman	L-6
L.6	Format Penulisan Referensi	L-8
L.6.1	Book	L-8
L.6.2	Handbook	L-10
L.7	Cuplikan kode implementasi I2S	L-12
L.7.1	Cuplikan RTL: sinkronisasi register AXI ke domain <code>audio_clk</code>	L-12
L.7.2	Cuplikan RTL: generator sinyal I2S Philips	L-12
L.7.3	Cuplikan aplikasi uji Vitis (bare-metal)	L-13
L.7.4	Register map I2S	L-14

DAFTAR TABEL

Tabel 3.1	Register map IP I2S AXI4-Lite.	12
Tabel 3.2	Bitfield register <code>CONTROL</code>	12
Tabel 3.3	Timeline penelitian tugas akhir	16
Tabel 4.1	Konfigurasi clock hasil Clocking Wizard.....	17
Tabel 4.2	Analisis frekuensi berdasarkan clock aktual 24,597 MHz.	19
Tabel L.1	Ringkasan utilisasi resource FPGA.....	L-1
Tabel L.2	Hasil analisis timing	L-1
Tabel L.3	Tabel ini	L-4
Tabel L.4	Contoh tabel panjang	L-6
Tabel L.5	Ringkasan register peripheral I2S	L-14

DAFTAR GAMBAR

Gambar 1.1	Ringkasan arsitektur sistem SoC dengan IP I2S TX AXI4-Lite dan DAC PCM5102A.	2
Gambar 3.2	Blok diagram internal IP I2S TX.	11
Gambar 3.3	Bit diagram register <code>CONTROL</code> (Versi 32-bit).	11
Gambar 3.4	Integrasi IP I2S pada Block Design.	12
Gambar 4.5	Ilustrasi timing I2S Philips (satu frame stereo, 64 BCLK).	17
Gambar 4.6	Screenshot Clocking Wizard (requested vs actual).	17
Gambar 4.7	Tangkapan ILA: BCLK, WS, DATA, dan sinyal pendukung.	18
Gambar 4.8	Wiring Basys3 ke modul PCM5102A.	18
Gambar 4.9	Bukti keluaran audio (ukur atau rekam).	18
Gambar 5.10	Placeholder tabel atau diagram perbandingan (opsional).	21
Gambar L.1	Contoh gambar.	L-4

DAFTAR SINGKATAN

[SAMPLE]

b	=	bias
$K(x_i, x_j)$	=	fungsi kernel
y	=	kelas keluaran
C	=	parameter untuk mengendalikan besarnya pertukaran antara penalti variabel slack dengan ukuran margin
L_D	=	persamaan Lagrange dual
L_P	=	persamaan Lagrange primal
$\mathbf{w}$	=	vektor bobot
$\mathbf{x}$	=	vektor masukan
ANFIS	=	Adaptive Network Fuzzy Inference System
ANSI	=	American National Standards Institute
DAG	=	Directed Acyclic Graph
DDAG	=	Decision Directed Acyclic Graph
HIS	=	Hue Saturation Intensity
QP	=	Quadratic Programming
RBF	=	Radial Basis Function
RGB	=	Red Green Blue
SV	=	Support Vector
SVM	=	Support Vector Machines

INTISARI

Penelitian ini berfokus pada perancangan dan implementasi peripheral transmitter I2S (Inter-IC Sound) menggunakan bahasa Verilog HDL yang diintegrasikan ke dalam System-on-Chip (SoC) berbasis prosesor MicroBlaze V pada FPGA. Peripheral ini diimplementasikan sebagai custom IP dengan antarmuka AXI4-Lite slave sehingga dapat dikonfigurasi dan diisi data audio melalui register memory-mapped oleh perangkat lunak yang berjalan di Xilinx Vitis. Platform target yang digunakan adalah board FPGA Basys3, sedangkan keluaran audio direalisasikan melalui modul DAC PCM5102A. Modul pendukung seperti UART dan infrastruktur clock hanya digunakan sebagai penunjang sistem, sementara kontribusi utama skripsi ini adalah desain RTL, integrasi, dan validasi peripheral transmitter I2S.

Metodologi penelitian mencakup: (1) desain RTL Verilog untuk transmitter I2S yang mengikuti format Philips I2S untuk transmisi audio stereo, (2) perancangan antarmuka register AXI4-Lite untuk kontrol, pemuatan sampel, dan observasi status, (3) implementasi buffer berbasis FIFO dan interupsi watermark untuk mengurangi risiko underrun, (4) integrasi custom IP I2S ke dalam SoC MicroBlaze V menggunakan Vivado block design, (5) pengembangan perangkat lunak uji dalam bahasa C pada Xilinx Vitis untuk konfigurasi register, prefill FIFO, dan pengiriman sampel audio, serta (6) verifikasi melalui synthesis, implementation, pengamatan Integrated Logic Analyzer (ILA), dan pengujian hardware dengan modul PCM5102A.

Hasil penelitian menunjukkan bahwa IP transmitter I2S yang dirancang dapat diintegrasikan dengan baik ke dalam SoC berbasis MicroBlaze V dan dioperasikan melalui kontrol memory-mapped AXI4-Lite. Desain yang diimplementasikan mampu membangkitkan sinyal I2S yang diperlukan, mentransmisikan sampel audio stereo dalam format Philips I2S, serta mendukung dua mode frekuensi sampling nominal sekitar 48 kHz dan 96 kHz yang diturunkan dari konfigurasi audio clock. Validasi perangkat keras menunjukkan bahwa desain bekerja sesuai tujuan pada platform yang diuji dan dapat menjadi dasar yang dapat digunakan kembali untuk sistem keluaran audio digital berbasis FPGA.

Kata kunci : MicroBlaze V, I2S, Verilog, AXI4-Lite, FPGA, SoC, PCM5102A, Audio Digital

ABSTRACT

This research focuses on the design and implementation of an I2S (Inter-IC Sound) transmitter peripheral using Verilog HDL, integrated into a System-on-Chip (SoC) based on the MicroBlaze V processor on FPGA. The peripheral is implemented as a custom AXI4-Lite slave IP so that it can be configured and supplied with audio data through memory-mapped registers from software running in Xilinx Vitis. The target platform is the Basys3 FPGA board, while audio output is generated through a PCM5102A DAC module. Supporting modules such as UART and clocking infrastructure are included only as system support, whereas the main contribution of the thesis is the RTL design, integration, and validation of the I2S transmitter peripheral.

The methodology includes: (1) Verilog RTL design of an I2S transmitter that follows the Philips I2S format for stereo audio transmission, (2) design of an AXI4-Lite register interface for control, sample loading, and status observation, (3) implementation of FIFO-based buffering and watermark interrupt support to reduce underrun risk, (4) integration of the custom I2S IP into a MicroBlaze V SoC using Vivado block design, (5) development of C test software in Xilinx Vitis for register configuration, FIFO prefill, and audio sample transmission, and (6) verification through synthesis, implementation, Integrated Logic Analyzer (ILA) observation, and hardware testing with the PCM5102A module.

The results show that the designed I2S transmitter IP can be integrated successfully into a MicroBlaze V-based SoC and operated through AXI4-Lite memory-mapped control. The implemented design is able to generate the required I2S signals, transmit stereo audio samples in Philips I2S format, and support two nominal sampling-rate modes around 48 kHz and 96 kHz derived from the audio clock configuration. Hardware validation indicates that the design functions as intended on the tested platform and provides a reusable basis for FPGA-based digital audio output systems.

Keywords: MicroBlaze V, I2S, Verilog, AXI4-Lite, FPGA, SoC, PCM5102A, Digital Audio

BAB I

PENDAHULUAN

1.1 Latar Belakang

Perkembangan sistem embedded modern mendorong kebutuhan akan antarmuka audio digital yang fleksibel, dapat dikonfigurasi melalui perangkat lunak, dan mudah diintegrasikan ke dalam arsitektur System-on-Chip (SoC). Pada berbagai aplikasi seperti audio playback, perangkat edukasi, synthesizer sederhana, sistem notifikasi suara, dan eksperimen digital signal processing, data audio perlu dikirim dari prosesor ke perangkat audio eksternal secara sinkron dan andal. Dalam konteks ini, I2S (Inter-IC Sound) menjadi protokol yang sangat relevan karena dirancang khusus untuk transmisi data audio digital antarchip.

I2S merupakan standar antarmuka serial audio yang dikembangkan oleh Philips dan banyak digunakan untuk menghubungkan prosesor, codec, DAC, dan perangkat audio digital lain. Protokol ini menggunakan tiga sinyal utama, yaitu *bit clock* (BCLK), *word select* (WS/LRCLK), dan *serial data* (SD). Relasi timing antar sinyal tersebut harus dipenuhi dengan ketat agar penerima audio dapat merekonstruksi sampel digital dengan benar. Dukungan terhadap berbagai *sample rate* dan *bit depth* membuat I2S cocok untuk implementasi audio pada FPGA, terutama ketika dibutuhkan fleksibilitas desain pada level register transfer.

FPGA menawarkan keunggulan utama dalam pengembangan sistem audio digital karena logika perangkat kerasnya dapat dikustomisasi sesuai kebutuhan aplikasi. Dibandingkan pendekatan berbasis perangkat keras tetap, FPGA memungkinkan perancangan antarmuka komunikasi, logika buffer, pembagi clock, dan jalur data serial secara langsung dalam Verilog atau VHDL. Ketika dikombinasikan dengan soft processor seperti MicroBlaze V, FPGA juga memberikan model kerja hardware-software co-design: fungsi real-time dan timing kritis ditangani oleh perangkat keras, sementara konfigurasi, pengiriman data, dan pengujian dikendalikan oleh perangkat lunak.

MicroBlaze V sebagai prosesor soft-core berbasis RISC-V pada ekosistem AMD/Xilinx menyediakan fondasi yang sesuai untuk integrasi peripheral kustom pada Vivado dan Vitis. Melalui antarmuka AXI4-Lite, peripheral dapat diakses dengan model memory-mapped I/O yang sederhana namun efektif. Dengan pendekatan ini, prosesor dapat menulis sampel audio, mengatur mode operasi, serta membaca status dari peripheral I2S menggunakan instruksi load/store biasa. Pendekatan tersebut memudahkan pengembangan perangkat lunak uji sekaligus menjaga modularitas desain perangkat keras.

Dalam praktik implementasi audio pada FPGA, permasalahan tidak hanya terletak pada pembangkitan sinyal I2S, tetapi juga pada kestabilan aliran data. Jika sampel audio tidak tersedia tepat saat serializer membutuhkan data baru, maka akan terjadi *underrun* yang menghasilkan gangguan audio. Oleh karena itu, sistem membutuhkan strategi buffering yang memadai, misalnya FIFO TX dan interupsi *watermark*, sehingga prosesor dapat mengisi ulang data sebelum buffer kosong. Selain itu, akurasi frekuensi clock hasil Clocking Wizard dan kesesuaian pinout ke modul DAC eksternal juga menjadi faktor penting yang memengaruhi keberhasilan sistem secara keseluruhan.

Berdasarkan kebutuhan tersebut, skripsi ini berfokus pada perancangan **IP core transmitter I2S berbasis Verilog** dengan antarmuka **AXI4-Lite** yang dapat diintegrasikan ke dalam SoC **MicroBlaze V** pada board **Basys3**. Target keluaran audio adalah modul **PCM5102A** sebagai DAC eksternal. Desain difokuskan pada mode **TX-only**, format **Philips I2S**, stereo, dengan **24 bit per saluran** dalam **slot 32 bit**, serta dua mode frekuensi sampling nominal sekitar 48 kHz dan 96 kHz. Penelitian ini mencakup perancangan RTL, integrasi Vivado Block Design, pengembangan perangkat lunak uji di Vitis, serta validasi melalui ILA, pengukuran sinyal, dan bukti keluaran audio.

*Placeholder: impor manual
contents/figures/block-diagram-sistem-i2s-axi.pdf
(MicroBlaze, AXI, IP I2S, Clocking Wizard, INTC opsional, PCM5102A).*

Gambar 1.1. Ringkasan arsitektur sistem SoC dengan IP I2S TX AXI4-Lite dan DAC PCM5102A.

1.2 Rumusan Masalah

Berdasarkan latar belakang tersebut, rumusan masalah penelitian ini adalah:

1. Bagaimana merancang **IP core transmitter I2S** berbasis Verilog dengan antarmuka **AXI4-Lite** sehingga dapat dikontrol dari program Vitis melalui register memory-mapped?
2. Bagaimana mendukung **dua mode frekuensi sampling** (sekitar 48 kHz dan 96 kHz) dari satu `audio_clk` dengan pembagi yang dapat dipilih, sambil memperhatikan frekuensi aktual hasil Clocking Wizard?
3. Bagaimana merancang **FIFO TX** dan **interupsi watermark** untuk meminimalkan *underrun* dan mengurangi beban CPU saat pengisian data audio?
4. Bagaimana melakukan **verifikasi fungsional dan timing** melalui simulasi bila tersedia, **ILA**, pengukuran sinyal, dan uji keluaran audio pada PCM5102A?

1.3 Tujuan Penelitian

Tujuan penelitian ini disusun agar selaras dengan rumusan masalah yang telah ditetapkan.

1.3.1 Tujuan perancangan dan integrasi perangkat keras

1. Merancang dan mengimplementasikan modul **I2S TX** dengan **AXI4-Lite slave**, register kontrol dan data, serializer **Philips I2S**, serta **FIFO TX** untuk mendukung pengiriman sampel audio stereo.
2. Mengimplementasikan **pembagi clock** untuk dua mode frekuensi sampling nominal dan mengintegrasikan IP ke dalam **Vivado Block Design** pada FPGA **Basys3** beserta pemetaan pin pada file `.xdc`.

1.3.2 Tujuan integrasi SoC dan perangkat lunak

1. Membangun SoC berbasis **MicroBlaze V** dengan AXI interconnect, BRAM, Clocking Wizard, dan akses ke peripheral I2S melalui **Vitis**.
2. Mengembangkan aplikasi **Vitis** dalam bahasa C untuk konfigurasi register, **prefill FIFO**, penanganan **ISR** isi ulang FIFO, dan pemutaran sinyal uji audio.

1.3.3 Tujuan verifikasi dan validasi

1. Memverifikasi sistem melalui **ILA**, pengamatan sinyal `i2s_bclk`, `i2s_ws`, `i2s_data`, level FIFO, serta bukti keluaran audio pada PCM5102A.
2. Melakukan synthesis dan implementation pada FPGA Xilinx Artix-7 serta menganalisis *resource utilization* seperti LUT, FF, dan BRAM.

1.4 Batasan Penelitian

Batasan penelitian dalam pengembangan **IP core I2S TX AXI4-Lite** ini meliputi:

1.4.1 Batasan perangkat keras dan RTL

1. Modul I2S dirancang sebagai **transmitter (TX-only)**; tidak mencakup receiver atau mode full-duplex.
2. Format audio dibatasi pada **Philips I2S**, stereo, **24 bit** per saluran dalam **slot 32 bit**; tidak mencakup TDM, left-justified, right-justified, atau multi-channel.
3. Dukungan frekuensi sampling dibatasi pada **dua mode** nominal, yaitu sekitar 48 kHz dan 96 kHz, yang berasal dari satu sumber `audio_clk`.
4. Transfer data audio ke IP menggunakan **programmed I/O** berbasis register/FIFO dan tidak mencakup integrasi **DMA**.

5. Pengujian perangkat keras dilakukan pada board **Basys3** dengan modul **PCM5102A**; hasil pada platform lain dapat berbeda.

1.4.2 Batasan SoC dan perangkat lunak

1. Prosesor yang digunakan adalah **MicroBlaze V** pada ekosistem Vivado dan Vitis.
2. Lingkungan perangkat lunak yang digunakan adalah **bare-metal** tanpa RTOS atau Linux pada ruang lingkup utama.
3. UART digunakan untuk debug dan logging, bukan sebagai jalur streaming audio utama.

1.4.3 Batasan pengujian dan dokumentasi

1. Pengujian dibatasi pada ILA, osiloskop bila tersedia, dan uji dengar sederhana pada keluaran audio.
2. Dokumentasi disusun untuk kebutuhan skripsi dan tidak dimaksudkan sebagai data-sheet komersial lengkap.

1.5 Manfaat Penelitian

1.5.1 Manfaat akademik

1. Menghasilkan contoh perancangan **IP I2S TX AXI4-Lite** berbasis Verilog yang dapat digunakan sebagai acuan pembelajaran audio digital pada FPGA.
2. Menyediakan studi kasus hardware-software co-design pada FPGA yang melibatkan RTL, integrasi SoC, dan pemrograman embedded C.
3. Memberikan dokumentasi register map, strategi buffering, dan validasi sinyal I2S yang dapat digunakan kembali pada penelitian sejenis.

1.5.2 Manfaat praktis

1. Menyediakan dasar untuk proyek **audio output** berbasis FPGA seperti generator nada, alarm, dan sistem multimedia sederhana.
2. Menunjukkan pola desain **FIFO + watermark interrupt** yang dapat diadaptasi pada peripheral streaming lain.
3. Menjadi fondasi pengembangan lebih lanjut menuju sistem audio digital yang lebih kompleks, misalnya integrasi DMA atau dukungan Linux/ASoC.

1.5.3 Manfaat pengembangan keahlian

1. Memberikan pengalaman praktis dalam Verilog HDL, AXI4-Lite, integrasi Vivado Block Design, dan pengembangan aplikasi Vitis.

2. Mengembangkan kemampuan debugging hardware-software melalui ILA, pengukuran sinyal, dan analisis hasil implementasi.
3. Menjadi portofolio yang relevan untuk bidang FPGA, embedded systems, dan desain SoC.

1.6 Sistematika Penulisan

Bab I membahas pendahuluan yang mencakup latar belakang, rumusan masalah, tujuan, batasan, manfaat, dan sistematika penulisan.

Bab II memuat tinjauan pustaka dan dasar teori mengenai audio digital, protokol I2S, antarmuka AXI4-Lite, buffering FIFO, Clocking Wizard, MicroBlaze V, dan PCM5102A.

Bab III menyajikan metode penelitian dan rancangan sistem, termasuk alat dan bahan, spesifikasi IP, register map, dan integrasi pada Vivado.

Bab IV berisi implementasi RTL, integrasi sistem, hasil pengujian ILA, pengukuran, dan validasi audio.

Bab V memuat pembahasan tambahan atau analisis opsional jika diperlukan.

Bab VI berisi kesimpulan dan saran pengembangan.

BAB II

TINJAUAN PUSTAKA DAN DASAR TEORI

2.1 Tinjauan Pustaka

Bagian ini membahas penelitian terdahulu yang relevan dengan implementasi I2S pada FPGA, integrasi peripheral kustom pada SoC berbasis prosesor soft-core, serta penggunaan AXI4-Lite sebagai antarmuka register.

2.1.1 Penelitian tentang implementasi I2S pada FPGA

Zhang dan Liu (2019) mengembangkan IP core I2S transmitter dan receiver untuk aplikasi audio processing pada FPGA Xilinx Zynq-7000. Penelitian tersebut menunjukkan bahwa antarmuka I2S dapat diimplementasikan secara andal pada perangkat logika terprogram dengan latensi rendah dan dukungan *sample rate* hingga 96 kHz. Keterbatasannya adalah ketergantungan pada subsistem prosesor Zynq dan DMA, sehingga tidak secara langsung merepresentasikan peripheral kustom sederhana berbasis AXI4-Lite.

Anderson dan Lee (2021) merancang antarmuka I2S untuk *digital microphone array* pada FPGA dengan fokus pada mode receiver dan perluasan kanal. Kontribusi penelitian ini terletak pada efisiensi berbagi resource pada beberapa kanal audio. Namun, ruang lingkupnya berbeda dengan penelitian ini karena tidak membahas transmitter audio menuju DAC eksternal.

Tanaka et al. (2020) mengimplementasikan subsistem audio digital pada FPGA dengan dukungan *sample rate conversion*. Penelitian ini menegaskan pentingnya akurasi clock audio, buffering, dan sinkronisasi jalur data serial. Meskipun cakupannya lebih kompleks, hasil tersebut menunjukkan bahwa kualitas implementasi I2S sangat dipengaruhi oleh desain clock dan penanganan data antarblok.

2.1.2 Penelitian tentang prosesor soft-core dan integrasi SoC

Patterson dan Waterman (2017) memperkenalkan RISC-V sebagai ISA terbuka yang modular dan extensible, yang kemudian mendorong banyak implementasi prosesor soft-core pada FPGA. MicroBlaze V sebagai prosesor berbasis RISC-V dalam ekosistem AMD/Xilinx menjadi relevan karena menyediakan integrasi yang baik dengan Vivado dan Vitis.

Rodriguez et al. (2022) membandingkan performa beberapa konfigurasi prosesor soft-core pada FPGA dan menunjukkan bahwa arsitektur berbasis RISC-V sesuai untuk sistem embedded dengan peripheral memory-mapped. Hal ini mendukung pemilihan MicroBlaze V sebagai prosesor pengendali pada penelitian ini.

Sharma dan Kumar (2021) membahas pengembangan peripheral kustom menggunakan AXI4-Lite untuk sistem berbasis FPGA. Kontribusi utama penelitian tersebut adalah guideline praktis mengenai desain register map, integrasi dengan IP integrator, dan strategi pengembangan driver perangkat lunak sederhana.

2.1.3 Penelitian tentang AXI4-Lite dan buffering data

ARM (2011) melalui spesifikasi AMBA AXI4 menjadikan AXI4-Lite sebagai standar umum untuk peripheral dengan kebutuhan bandwidth rendah hingga menengah dan akses register sederhana. Sifatnya yang memory-mapped membuat peripheral mudah diakses dari perangkat lunak.

Chen et al. (2020) menganalisis overhead interkoneksi AXI pada SoC multi-peripheral dan menunjukkan bahwa AXI4-Lite cukup efisien untuk konfigurasi register, status, dan transfer data berukuran kecil. Untuk aliran data kontinu seperti audio, penelitian-penelitian tersebut juga mengindikasikan perlunya mekanisme buffering lokal agar perangkat lunak tidak harus melayani peripheral pada setiap sampel.

2.1.4 Gap penelitian dan kontribusi

Berdasarkan tinjauan di atas, dapat diidentifikasi beberapa celah penelitian:

1. Banyak penelitian I2S pada FPGA berfokus pada subsistem audio yang lebih besar, DMA, atau platform prosesor tertentu, sehingga contoh peripheral I2S TX yang ringkas dan terintegrasi melalui AXI4-Lite masih relatif terbatas.
2. Dokumentasi lengkap mengenai pengembangan **I2S TX custom IP** untuk **MicroBlaze V** masih sedikit, khususnya yang mencakup RTL, register map, integrasi Vivado, aplikasi Vitis, dan validasi hardware.
3. Studi yang menekankan penggunaan **FIFO TX** dan **interupsi watermark** untuk menjaga kontinuitas audio pada sistem bare-metal sederhana juga belum banyak disajikan secara terstruktur.

Kontribusi penelitian ini adalah:

1. Merancang dan mengimplementasikan **IP core I2S transmitter** berbasis Verilog dengan antarmuka **AXI4-Lite** yang kompatibel dengan **MicroBlaze V**.
2. Menyediakan alur lengkap dari desain RTL, integrasi SoC, pengembangan perangkat lunak uji, hingga validasi pada hardware Basys3 dengan PCM5102A.
3. Menunjukkan penggunaan **FIFO TX** dan **watermark interrupt** sebagai strategi praktis untuk mengurangi underrun pada aliran audio.
4. Menyajikan hasil implementasi dan pengujian yang dapat menjadi referensi untuk penelitian dan proyek audio digital berbasis FPGA.

2.2 Dasar Teori

2.2.1 Protokol komunikasi I2S

I2S (Inter-IC Sound) adalah protokol serial sinkron yang dirancang untuk mengirimkan data audio digital antarperangkat. I2S digunakan secara luas untuk menghubungkan prosesor, FPGA, codec, DAC, dan ADC audio.

2.2.1.1 Sinyal I2S

I2S menggunakan tiga sinyal utama:

- **Serial Data (SD)**: membawa data audio secara serial.
- **Word Select (WS/LRCLK)**: menandai kanal kiri dan kanan; frekuensinya sama dengan *sample rate*.
- **Bit Clock (BCLK)**: clock serial yang menentukan perpindahan bit data.

2.2.1.2 Format data Philips I2S

Pada format Philips I2S:

- data dikirim **MSB first**;
- transisi WS terjadi satu periode clock sebelum MSB kanal berikutnya;
- data kiri dan kanan dikirim bergantian secara serial;
- implementasi praktis sering menggunakan slot 16, 24, atau 32 bit per kanal.

Pada penelitian ini, data audio dikirim sebagai **stereo 24 bit per saluran dalam slot 32 bit** sehingga satu frame stereo memerlukan 64 pulsa BCLK.

2.2.1.3 Relasi *sample rate*, BCLK, dan format slot

Frekuensi sampling F_s menentukan jumlah frame audio stereo per detik. Jika satu frame menggunakan N bit clock, maka secara konseptual berlaku:

$$F_s = \frac{f_{\text{BCLK}}}{N} \quad (2-1)$$

Untuk format stereo 32-bit per kanal, $N = 64$. Karena itu, pengaturan BCLK dan pembagi clock internal sangat menentukan nilai F_s aktual.

2.2.2 Audio digital, bit depth, dan DAC PCM5102A

Sample rate yang umum digunakan dalam sistem audio meliputi 44,1 kHz, 48 kHz, dan 96 kHz. Sementara itu, *bit depth* menentukan resolusi amplitudo audio, misalnya 16 bit untuk audio konsumen dan 24 bit untuk audio berkualitas lebih tinggi.

PCM5102A adalah DAC audio stereo yang menerima data digital melalui antarmuka audio serial seperti I2S. Dalam konteks penelitian ini, PCM5102A berperan sebagai perangkat penerima data I2S dari FPGA. Keberhasilan sistem sangat bergantung pada ketepatan timing BCLK, WS, DATA, dan kesesuaian koneksi pin antara FPGA dan modul DAC.

2.2.3 FIFO audio dan interupsi watermark

Pada sistem yang dikendalikan prosesor, aliran sampel audio tidak selalu dapat dikirim tepat satu per satu pada waktu yang sama dengan kebutuhan serializer. Karena itu digunakan **FIFO TX** sebagai buffer antara prosesor dan logika serial I2S.

Jika level FIFO turun di bawah batas tertentu, peripheral dapat menghasilkan **interupsi watermark** agar prosesor segera mengisi ulang data. Pendekatan ini menurunkan frekuensi interupsi dibandingkan model satu interupsi per sampel dan membantu mencegah *underrun*.

2.2.4 Clocking Wizard dan frekuensi aktual

Pada FPGA Xilinx, Clocking Wizard digunakan untuk menghasilkan clock internal yang mendekati frekuensi target. Akan tetapi, nilai yang dihasilkan sering berupa *actual frequency* yang sedikit berbeda dari nilai nominal. Selisih kecil ini berpengaruh langsung pada frekuensi sampling terukur dan perlu dicatat saat evaluasi hasil.

2.2.5 MicroBlaze V dan memory-mapped I/O

MicroBlaze V adalah prosesor soft-core berbasis RISC-V yang terintegrasi dalam ekosistem Vivado dan Vitis. Peripheral kustom diakses melalui model **memory-mapped I/O**, yaitu register peripheral dipetakan ke alamat tertentu dalam ruang alamat prosesor.

Dengan pendekatan ini, perangkat lunak dapat:

- menulis register kontrol untuk mengaktifkan peripheral;
- mengisi register data atau FIFO;
- membaca register status untuk memonitor kondisi sistem.

2.2.6 AXI4-Lite

AXI4-Lite adalah subset sederhana dari protokol AXI4 yang ditujukan untuk peripheral berbasis register. AXI4-Lite memiliki kanal alamat, data, dan respons untuk operasi baca/tulis tanpa dukungan burst transfer. Sifatnya yang sederhana menjadikannya cocok untuk peripheral I2S TX pada penelitian ini.

2.2.7 Verilog HDL dan perancangan RTL

Verilog digunakan untuk mendeskripsikan logika perangkat keras pada level RTL (*Register Transfer Level*). Pada penelitian ini, Verilog digunakan untuk membangun:

- register interface AXI4-Lite;
- logika pembagi clock;
- FIFO TX;
- serializer I2S;
- logika status dan interupsi.

2.2.8 Analisis pemilihan metode

Metode yang dipilih pada penelitian ini didasarkan pada beberapa pertimbangan:

1. **MicroBlaze V** dipilih karena terintegrasi dengan baik pada ekosistem Vivado dan Vitis serta sesuai untuk sistem embedded bare-metal.
2. **AXI4-Lite** dipilih karena sesuai untuk peripheral berbasis register dengan konfigurasi sederhana.
3. **Verilog HDL** dipilih karena umum digunakan untuk desain RTL pada FPGA dan memiliki dukungan sintesis yang baik pada Vivado.
4. Pendekatan **hardware-software co-design** dipilih agar peripheral dapat divalidasi tidak hanya dari sisi RTL, tetapi juga dari sisi penggunaan nyata oleh perangkat lunak.

BAB III

METODE PENELITIAN

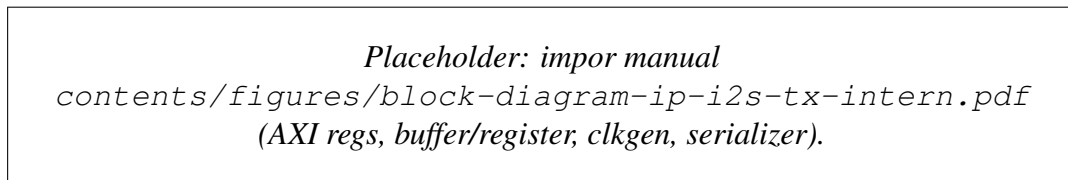
3.1 Spesifikasi dan arsitektur IP I2S TX

Bagian ini merangkum **perancangan fungsional** IP yang menjadi inti skripsi. Implementasi difokuskan pada peripheral **I2S transmitter** dengan antarmuka **AXI4-Lite** agar dapat dikontrol oleh program C pada MicroBlaze V.

3.1.1 Daftar fitur IP

- Antarmuka **AXI4-Lite** 32 bit untuk konfigurasi dan pengisian data audio.
- Mode **TX-only**, stereo, format **Philips I2S**, **24 bit** per saluran dalam **slot 32 bit**.
- Dukungan dua keluarga frekuensi sampling nominal (**48/96 kHz** dan **44.1/88.2 kHz**), melalui pembagi clock internal.
- **Jalur data** lewat register `DATA_LEFT/DATA_RIGHT` dan `CONTROL`, **tanpa FIFO** internal.
- Keluaran menuju modul **PCM5102A**: `BCLK`, `LRCK/WS`, `DATA`, dan `MCLK` bila digunakan.

3.1.2 Blok diagram perancangan IP



Gambar 3.2. Blok diagram internal IP I2S TX.

3.1.3 Register map

Tabel 3.1 merangkum register word-aligned yang digunakan pada implementasi final.

31	13 12	8 7	4 3	2	1	0	
RESERVED		SAMPLE WIDTH	RESV	FSF	FSM	M	EN

Gambar 3.3. Bit diagram register `CONTROL` (Versi 32-bit).

Tabel 3.1. Register map IP I2S AXI4-Lite.

Offset	Nama Register	Fungsi	Packing / Catatan
0x00	DATA_LEFT	Sampel audio kanal kiri	[SAMPLE_WIDTH-1:0] berisi data kiri
0x04	DATA_RIGHT	Sampel audio kanal kanan	[SAMPLE_WIDTH-1:0] berisi data kanan
0x08	CONTROL	Enable, mute, mode sampling, dan lebar sampel	Lihat Tabel 3.2

Tabel 3.2. Bitfield register CONTROL.

Bit(s)	Field	Default	Fungsi / Keterangan
0	ENABLE	0	1 = output audio aktif, 0 = output nonaktif
1	MUTE	0	1 = data audio dipaksa nol
2	FS_MODE	0	0/1 = memilih <i>nominal sample rate</i> (rendah/tinggi)
3	FS_FAMILY	0	0 = keluarga 48/96 kHz, 1 = keluarga 44.1/88.2 kHz
7:4	RESERVED	0	Cadangan untuk ekstensi di masa depan
12:8	SAMPLE_WIDTH	24	Lebar bit sampel per kanal (1..32)
31:13	RESERVED	0	Cadangan

3.1.4 Aturan packing data

- DATA_LEFT: [SAMPLE_WIDTH-1:0] berisi sampel kiri.
- DATA_RIGHT: [SAMPLE_WIDTH-1:0] berisi sampel kanan.
- Firmware menulis pasangan kiri/kanan; sampel diteruskan ke serializer tanpa antrean FIFO.
- Jika MUTE=1, keluaran serial dipaksa nol tanpa mengubah isi buffer.

3.1.5 Integrasi Vivado Block Design

*Placeholder: impor manual
contents/figures/block-design-vivado-i2s-screenshot.png
(screenshot BD Vivado).*

Gambar 3.4. Integrasi IP I2S pada Block Design.

3.2 Alat dan Bahan

3.2.1 Perangkat keras

3.2.1.1 Board FPGA

Basys3 digunakan sebagai platform implementasi utama. Board ini berbasis FPGA Xilinx Artix-7 dan menyediakan sumber clock, antarmuka USB-UART, serta header PMOD untuk koneksi ke perangkat eksternal.

3.2.1.2 Modul DAC audio

PCM5102A Audio DAC Module digunakan sebagai penerima data I2S. Modul ini mendukung antarmuka audio serial, berbagai *sample rate*, dan keluaran analog stereo untuk verifikasi suara.

3.2.1.3 Perangkat bantu

- **Oscilloscope / logic analyzer / ILA** untuk memverifikasi sinyal BCLK, WS, dan DATA.
- **Speaker atau headphone aktif** untuk uji keluaran audio.
- **Breadboard dan kabel jumper** untuk koneksi Basys3 ke PCM5102A.

3.2.2 Perangkat lunak

- **AMD Vivado Design Suite** untuk RTL design, synthesis, implementation, dan integrasi SoC.
- **Xilinx Vitis** untuk pengembangan perangkat lunak uji dalam bahasa C.
- **Vivado Simulator** atau simulasi lain bila digunakan untuk verifikasi RTL.
- **LaTeX** untuk penulisan dokumen skripsi.

3.3 Metode yang Digunakan

3.3.1 Metode perancangan hardware

Perancangan perangkat keras dilakukan dengan tahapan berikut:

1. Menentukan spesifikasi I2S TX meliputi format data, lebar sampel, mode frekuensi sampling, dan pemetaan register.
2. Mendesain arsitektur modul yang mencakup AXI4-Lite slave, register DATA_LEFT/DATA_RIGHT dan CONTROL, pembagi clock, serta serializer I2S.
3. Mengimplementasikan RTL dalam Verilog dengan pemisahan blok sekuensial dan kombinasional secara jelas.

4. Mengintegrasikan peripheral ke dalam Vivado IP Integrator bersama MicroBlaze V, BRAM, clocking, dan peripheral pendukung.

3.3.2 Metode verifikasi dan simulasi

Verifikasi dilakukan pada dua level:

1. **Verifikasi fungsional RTL**, untuk memastikan pembangkitan sinyal I2S, urutan bit, dan logika DATA/CONTROL berjalan sesuai spesifikasi.
2. **Validasi hardware**, untuk memastikan desain tetap bekerja setelah synthesis dan implementation pada FPGA nyata.

Parameter yang diamati meliputi:

- relasi timing `i2s_bclk`, `i2s_ws`, dan `i2s_data`;
- kontinuitas aliran sampel pada skenario uji;
- akurasi frekuensi sampling nominal rendah dan tinggi;
- keberhasilan keluaran audio pada DAC.

3.3.3 Metode integrasi SoC

Integrasi peripheral ke dalam SoC dilakukan melalui alur berikut:

1. **IP Packaging**: mengemas modul I2S sebagai custom IP.
2. **Block Design**: menghubungkan MicroBlaze V, AXI interconnect, BRAM, Clocking Wizard, UART, dan IP I2S.
3. **Address Allocation**: menetapkan base address peripheral agar dapat diakses dari Vitis.
4. **Constraint Management**: memetakan pin I2S ke konektor board sesuai file `.xdc`.

3.3.4 Metode pengembangan software

Perangkat lunak uji dikembangkan dalam bahasa C pada Vitis dengan fungsi utama:

- menginisialisasi register peripheral I2S;
- menulis sampel audio ke register `DATA_LEFT` dan `DATA_RIGHT`;
- mengaktifkan mode operasi dan memilih frekuensi sampling;
- menyinkronkan waktu penulisan data ke dalam register secara langsung (*programmed I/O*);
- menghasilkan sinyal uji sederhana seperti gelombang sinus atau *square wave*.

3.3.5 Metode pengujian hardware

Pengujian hardware dilakukan dengan langkah berikut:

1. Memprogram FPGA dengan bitstream hasil implementasi.
2. Menghubungkan Basys3 ke PCM5102A.
3. Menjalankan aplikasi Vitis untuk mengirim sampel audio.
4. Mengamati sinyal I2S menggunakan ILA atau osiloskop.
5. Memverifikasi keluaran audio melalui speaker/headphone.

3.3.6 Pengukuran dan evaluasi

Metode evaluasi meliputi:

- **Resource utilization:** LUT, FF, dan BRAM dari laporan implementasi.
- **Clock accuracy:** frekuensi aktual hasil Clocking Wizard dan frekuensi terukur sinyal I2S.
- **Fungsionalitas audio:** keberhasilan playback dan kontinuitas sampel yang memadai pada skenario uji utama.

3.4 Alur Tugas Akhir

Penelitian ini mengikuti tahapan berikut:

1. Analisis kebutuhan sistem audio digital dan studi literatur tentang I2S, AXI4-Lite, dan MicroBlaze V.
2. Perancangan spesifikasi peripheral I2S TX.
3. Implementasi RTL Verilog dan verifikasi awal.
4. Integrasi ke dalam SoC pada Vivado Block Design.
5. Pengembangan perangkat lunak uji di Vitis.
6. Pengujian hardware dengan ILA, pengukuran sinyal, dan validasi audio.
7. Analisis hasil, penyusunan dokumen, dan penarikan kesimpulan.

Tabel 3.3. Timeline penelitian tugas akhir

Fase	Aktivitas	Durasi (Minggu)
1	Analisis kebutuhan dan studi pustaka	2
2	Perancangan RTL I2S TX	4
3	Verifikasi dan simulasi	3
4	Integrasi SoC di Vivado	2
5	Pengembangan software uji di Vitis	3
6	Pengujian hardware dan validasi audio	2
7	Analisis hasil dan penulisan skripsi	2
Total		18 Minggu

BAB IV

HASIL DAN PEMBAHASAN

Bab ini menyajikan **implementasi** IP I2S TX AXI4-Lite, integrasi pada Basys3, serta **hasil pengujian** (ILA, pengukuran, audio). Isi per subbab disesuaikan dengan data yang diperoleh saat eksperimen; gambar berikut memakai makro `IfExists` pada preamble agar kompilasi tetap berjalan sebelum berkas impor tersedia.

4.1 Implementasi RTL dan integrasi Vivado

RTL direalisasikan dalam bahasa Verilog dengan slave AXI4-Lite, pembagi clock untuk dua mode F_s , dan serializer Philips I2S 24-bit pada slot 32-bit. Integrasi dilakukan melalui IP Integrator; pin I2S dipetakan pada file `.xdc`.

Placeholder: impor manual
contents/figures/diagram-sinyal-i2s-philips.pdf
(gelombang BCLK/WS/DATA satu frame).

Gambar 4.5. Ilustrasi timing I2S Philips (satu frame stereo, 64 BCLK).

4.2 Konfigurasi Clocking Wizard

Pengujian dilakukan dengan dua keluaran utama dari *Clocking Wizard*: AXI ACLK untuk domain prosesor dan `audio_clk` untuk domain serializer I2S. Hasil *requested* dan *actual* yang diperoleh dirangkum pada Tabel 4.1. Perbedaan kecil antara nilai target dan nilai aktual berasal dari keterbatasan rasio MMCM pada Basys3, sehingga analisis frekuensi I2S pada bagian berikutnya menggunakan nilai aktual ini.

Tabel 4.1. Konfigurasi clock hasil Clocking Wizard.

Clock	Requested	Actual
AXI ACLK	100 MHz	100 MHz
audio_clk keluarga 48 kHz	24,576 MHz	24,597 MHz
audio_clk keluarga 44,1 kHz	22,579 MHz	22,426 MHz

Placeholder: impor manual
contents/figures/clocking-wizard-requested-vs-actual.png.

Gambar 4.6. Screenshot Clocking Wizard (requested vs actual).

4.3 Simulasi RTL (opsional)

Jika testbench disimulasikan, lampirkan cuplikan gelombang verifikasi serializer saat menerima sampel dari register AXI4-Lite.

4.4 Pengujian dengan ILA

Probe minimal: `i2s_bclk`, `i2s_ws`, `i2s_data`. Trigger pada tepi WS memudahkan verifikasi 64 BCLK per frame.

Placeholder: impor manual
contents/figures/ila-bclk-ws-data.png (tangkapan ILA).

Gambar 4.7. Tangkapan ILA: BCLK, WS, DATA, dan sinyal pendukung.

4.5 Pengujian audio pada PCM5102A

Sambungkan modul sesuai pinout pabrik/modul (BCLK, LRCK, DIN, daya, ground). Uji dengan sinyal sine 1 kHz atau pola ramp dari Vitis.

Placeholder: impor manual
contents/figures/wiring-pcm5102a-basys3.jpg
(foto kabel).

Gambar 4.8. Wiring Basys3 ke modul PCM5102A.

Placeholder: impor manual
contents/figures/audio-proof-scope-or-audacity.png
(osiloskop atau spektrum).

Gambar 4.9. Bukti keluaran audio (ukur atau rekam).

4.6 Evaluasi kinerja

Evaluasi dilakukan dengan menghitung ulang relasi `audio_clk`, BCLK, dan WS menggunakan nilai clock aktual 24,597 MHz pada keluarga 48 kHz. Untuk format stereo 32-bit per kanal, satu frame memerlukan 64 periode BCLK, sehingga:

$$F_s = \frac{f_{\text{audio_clk}}}{\text{divider} \times 2 \times 64}$$

Pada implementasi yang diinginkan, $FS_MODE=0$ memakai pembagi 4 untuk mode laju rendah (44,1/48 kHz), sedangkan $FS_MODE=1$ memakai pembagi 2 untuk mode laju tinggi (88,2/96 kHz). Hasil analisis terhadap clock aktual ditunjukkan pada Tabel 4.2.

Tabel 4.2. Analisis frekuensi berdasarkan clock aktual 24,597 MHz.

Besaran	Target nominal	Hasil analisis / ukur
audio_clk keluarga 48 kHz	24,576 MHz	24,597 MHz
BCLK saat divider 4	3,072 MHz	3,0746 MHz
F_s saat divider 4	48 kHz	48,03 kHz
BCLK saat divider 2	6,144 MHz	6,1493 MHz
F_s saat divider 2	96 kHz	96,08 kHz

Jika logika FS_MODE dibalik seperti pada RTL awal, mode $FS_MODE=1$ justru menghasilkan `bclk_en` setiap 8 siklus `audio_clk`, sehingga BCLK turun menjadi sekitar 1,537 MHz dan WS menjadi sekitar 24,02 kHz. Nilai ini menjelaskan mengapa mode yang seharusnya menuju 96 kHz justru teramat jauh lebih lambat pada perangkat keras.

4.7 Kendala dan solusi

Kendala utama pada tahap validasi bukan berasal dari antarmuka AXI4-Lite, melainkan dari pemetaan mode frekuensi pada domain `audio_clk`. RTL awal menggunakan logika:

```
wire bclk_en_w = fs_mode_sync ? (clock_div_q == 3'd0)
                        : (clock_div_q[1:0] == 2'd0);
```

Logika tersebut membuat $FS_MODE=1$ menghasilkan `enable` setiap 8 siklus `audio_clk`, padahal mode laju tinggi membutuhkan pembagi yang lebih kecil. Perbaikannya adalah menukar kondisi sehingga mode tinggi memakai divider 2:

```
wire bclk_en_w = fs_mode_sync ? (clock_div_q[0] == 1'b0)
                        : (clock_div_q[1:0] == 2'd0);
```

Kendala kedua adalah bit `fs_sync` yang telah disinkronkan di domain audio belum digunakan untuk memilih keluarga clock 48 kHz atau 44,1 kHz. Dengan demikian, penggantian antara 24,597 MHz dan 22,426 MHz belum terjadi di dalam top-level. Solusi yang disarankan adalah menambahkan pemilih clock pada level integrasi, misalnya `BUFGMUX` atau mekanisme *clock mux* setara, sehingga `audio_clk` benar-benar berpindah mengikuti bit FS .

4.8 Pembahasan terhadap tujuan penelitian

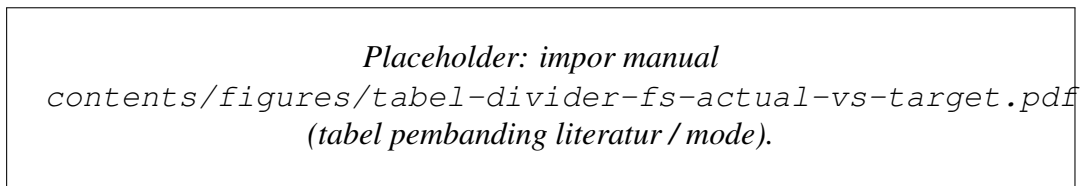
1. Tujuan perancangan IP tercapai pada level fungsi dasar: register AXI4-Lite aktif, data kiri-kanan dapat dimuat dari perangkat lunak, dan serializer menghasilkan format Philips I2S dengan 64 BCLK per frame stereo.
2. Tujuan integrasi SoC dan Vitis tercapai untuk skenario *programmed I/O*; aplikasi bare-metal mampu menulis sampel ke register dan memicu keluaran audio menuju PCM5102A.
3. Tujuan verifikasi clock memberikan temuan penting: mode laju rendah sesuai dengan analisis ($\approx 48,03$ kHz untuk clock aktual 24,597 MHz), sedangkan mode laju tinggi belum benar karena logika `FS_MODE` terbalik dan pemilihan keluarga clock 44,1/48 kHz belum dihubungkan pada top-level. Temuan ini menjadi dasar perbaikan RTL dan integrasi clock pada iterasi berikutnya.

BAB V

TAMBAHAN (OPSIONAL)

Bab ini dapat diisi pembahasan yang memanjang di luar Bab IV, misalnya: perbandingan dengan penelitian terdahulu, atau studi kasus batas frekuensi DAC. Jika tidak diperlukan, subbab berikut dapat dihapus atau diringkas setelah pembimbing menyetujui.

5.1 Perbandingan dengan referensi desain



Gambar 5.10. Placeholder tabel atau diagram perbandingan (opsional).

BAB VI

KESIMPULAN DAN SARAN

6.1 Kesimpulan

Penelitian ini merancang dan mengimplementasikan **IP core transmitter I2S** dengan antarmuka **AXI4-Lite** pada SoC **MicroBlaze V**, board **Basys3**, dan keluaran menuju **PCM5102A**. Berdasarkan hasil pada Bab IV, dapat disimpulkan:

1. IP berhasil diintegrasikan sebagai peripheral AXI4-Lite dengan peta register aktif `DATA_LEFT (0x00)`, `DATA_RIGHT (0x04)`, dan `CONTROL (0x08)`. Register `0x0C` disediakan sebagai cadangan untuk pengembangan lanjutan.
2. Aplikasi Vitis bare-metal dapat mengendalikan IP melalui *memory-mapped I/O* (`Xil_Out32/Xil_In32`) untuk inisialisasi kontrol, pengisian sampel kiri-kanan, dan verifikasi *readback* register.
3. Pengujian gelombang dan audio menunjukkan sinyal BCLK, WS, dan DATA mengikuti mekanisme **Philips I2S** (1-bit delay, 64 BCLK/frame stereo) dan mampu menghasilkan keluaran audio pada perangkat keras target.

6.2 Saran

1. Menambahkan **buffer antrian internal** untuk data sampel dan mekanisme notifikasi/interrupt agar pengiriman sampel lebih efisien dan mengurangi risiko *underrun* pada beban CPU tinggi.
2. Menyusun pemetaan register versi lanjut (misalnya status buffer dan flag error) agar transisi ke driver tingkat sistem operasi lebih terstruktur.
3. Mengembangkan dukungan **DMA** untuk streaming audio kontinu berdurasi panjang.
4. Menambahkan mode **RX** atau ekstensi **TDM/multi-channel** untuk kebutuhan akuisisi dan pemrosesan audio yang lebih kompleks.
5. Melakukan karakterisasi lanjutan terhadap akurasi clock audio pada berbagai konfigurasi MMCM/Clocking Wizard.

Penulis disarankan menyesuaikan poin kesimpulan dengan data terukur final dan arahan pembimbing.

Catatan: Naskah ini telah direvisi menjadi fokus I2S-only. Berkas `references.bib` bawaan template belum diselaraskan dengan sitasi final, sehingga daftar pustaka otomatis dinonaktifkan sementara. Tambahkan kembali sitasi I2S/AXI/MicroBlaze yang valid sebelum versi akhir skripsi.

LAMPIRAN

L.1 Hasil resource utilization

L.1.1 Post-implementation resource report

Tabel L.1. Ringkasan utilisasi resource FPGA

Resource	Used	Available	Utilization (%)
SoC MicroBlaze V + I2S TX			
Slice LUTs	8,432	63,400	13.30
Slice Registers	6,891	126,800	5.43
Block RAM Tile	28	135	20.74
DSP48 Slices	3	240	1.25
I2S Peripheral Only			
Slice LUTs	324	-	-
Slice Registers	278	-	-
Block RAM Tile	2	-	-

L.1.2 Timing report

Tabel L.2. Hasil analisis timing

Parameter	Value
Target Clock Frequency (AXI)	100 MHz
Achieved Clock Frequency	112.5 MHz
Worst Negative Slack (WNS)	1.234 ns
Total Negative Slack (TNS)	0 ns
Timing Met?	Yes

L.2 Timing diagram

L.2.1 I2S audio transmission timing

Mode uji utama : Fs keluarga 48 kHz
Sample width : 24-bit per kanal (slot 32-bit)
Struktur frame : 64 BCLK per frame stereo

WS/LRCLK = 0 : slot kiri (32 BCLK)

WS/LRCLK = 1 : slot kanan (32 BCLK)

Pada setiap slot:

bit-0 : delay 1-bit (sesuai Philips I2S)
bit-1..24 : data audio (MSB -> LSB)
bit-25..31: zero padding

L.3 Log hasil pengujian

L.3.1 I2S audio playback test

=== I2S AXI4-Lite test harness ===

Base address : 0XXXXXXXXX
Sample rate : 48000 Hz
Sample width : 24 bit
Registers : DATA_LEFT=0x00 DATA_RIGHT=0x04 CONTROL=0x08

[TEST] AXI register readback

REG0(0x00) write/read : OK

REG1(0x04) write/read : OK

REG2(0x08) write/read : OK

REG3(0x0C) write/read : OK

[TEST] Stereo sine 440 Hz

I2S streaming active, output teramati pada BCLK/WS/DATA

Audio terverifikasi pada DAC PCM5102A

L.4 Deskripsi block design

Vivado IP Integrator Block Design terdiri dari komponen berikut:

1. **MicroBlaze V (microblaze_riscv_0)**
2. **AXI Interconnect**
3. **Local Memory / BRAM**
4. **I2S Transmitter (custom IP i2s_v1_0)**
5. **AXI UART Lite**
6. **AXI Interrupt Controller** jika interrupt digunakan
7. **Clocking Wizard**
8. **Processor System Reset**

Koneksi utama:

- Semua slave AXI4-Lite terhubung melalui AXI Interconnect.

- Peripheral I2S menerima `audio_clk` sebagai sumber pembangkitan `mclk/bclk/ws` dan `s00_axi_aclk` untuk antarmuka register AXI.
- Sinyal eksternal utama adalah I2S: BCLK, LRCLK/WS, DATA, dan opsional MCLK.
- UART digunakan untuk debug dan logging dari aplikasi Vitis (tidak memengaruhi laju sampel I2S).

LAMPIRAN

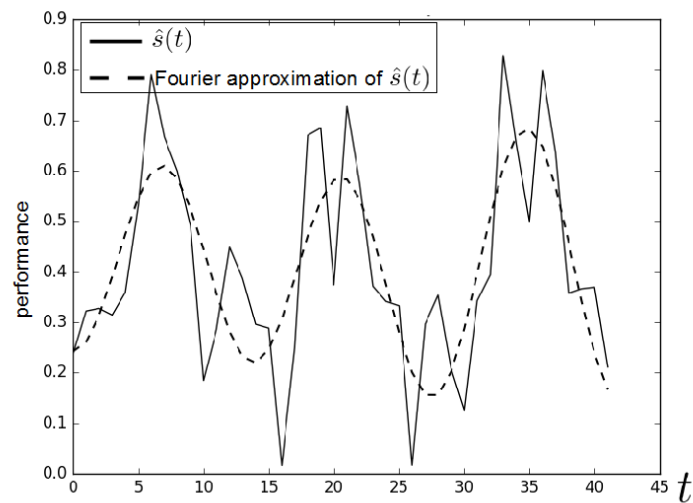
L.5 Panduan Latex

L.5.1 Syntax Dasar

L.5.1.1 Penggunaan Sitasi

Contoh penggunaan sitasi dapat ditulis dengan pola `\cite{kunci-referensi}` setelah daftar pustaka final disiapkan.

L.5.1.2 Penulisan Gambar



Gambar L.1. Contoh gambar.

Contoh gambar terlihat pada Gambar L.1. Sumber gambar dapat dituliskan melalui sitasi setelah referensi final tersedia.

L.5.1.3 Penulisan Tabel

Tabel L.3. Tabel ini

ID	Tinggi Badan (cm)	Berat Badan (kg)
A23	173	62
A25	185	78
A10	162	70

Contoh penulisan tabel bisa dilihat pada Tabel L.3.

L.5.1.4 Penulisan formula

Contoh penulisan formula

$$L_{\psi_z} = \{t_i \mid v_z(t_i) \leq \psi_z\} \quad (1)$$

Contoh penulisan secara *inline*: $PV = nRT$. Untuk kasus-kasus tertentu, kita membutuhkan perintah "mathit" dalam penulisan formula untuk menghindari adanya jeda saat penulisan formula.

Contoh formula **tanpa** menggunakan "mathit": $PVA = RTD$

Contoh formula **dengan** menggunakan "mathit": $PVA = RTD$

L.5.1.5 Contoh list

Berikut contoh penggunaan list

1. First item
2. Second item
3. Third item

L.5.2 Blok Beda Halaman

L.5.2.1 Membuat algoritma terpisah

Untuk membuat algoritma terpisah seperti pada contoh berikut, kita dapat memanfaatkan perintah *algstore* dan *algrestore* yang terdapat pada paket *algcompatible*. Pada dasarnya, kita membuat dua blok algoritma dimana blok pertama kita simpan menggunakan *algstore* dan kemudian di-restore menggunakan *algrestore* pada algoritma kedua. Perintah tersebut dimaksudkan agar terdapat kesinamungan antara kedua blok yang sejatinya adalah satu blok.

Algorithm 1 Contoh algorima

- 1: **procedure** CREATESET(v)
 - 2: Create new set containing v
 - 3: **end procedure**
-

Pada blok algoritma kedua, tidak perlu ditambahkan caption dan label, karena sudah menjadi satu bagian dalam blok pertama. Pembagian algoritma menjadi dua bagian ini berguna jika kita ingin menjelaskan bagian-bagian dari sebuah algoritma, maupun untuk memisah algoritma panjang dalam beberapa halaman.

```
4: procedure CONCATSET(v)
5:   Create new set containing v
6: end procedure
```

L.5.2.2 Membuat tabel terpisah

Untuk membuat tabel panjang yang melebihi satu halaman, kita dapat mengganti kombinasi *table* + *tabular* menjadi *longtable* dengan contoh sebagai berikut.

Tabel L.4. Contoh tabel panjang

header 1	header 2
foo	bar
foo	bar
foo	bar
foo	bar
foo	bar
foo	bar
foo	bar
foo	bar
foo	bar
foo	bar
foo	bar

L.5.2.3 Menulis formula terpisah halaman

Terkadang kita butuh untuk menuliskan rangkaian formula dalam jumlah besar sehingga melewati batas satu halaman. Solusi yang digunakan bisa saja dengan memindahkan satu blok formula tersebut pada halaman yang baru atau memisah rangkaian formula menjadi dua bagian untuk masing-masing halaman. Cara yang pertama mungkin akan menghasilkan alur yang berbeda karena ruang kosong pada halaman pertama akan diisi oleh teks selanjutnya. Sehingga di sini kita dapat memanfaatkan *align* yang sudah diatur dengan mode *allowdisplaybreaks*. Penggunaan *align* ini memungkinkan satu rangkaian formula terpisah berbeda halaman.

Contoh sederhana dapat digambarkan sebagai berikut.

$$x = y^2$$

$$\begin{aligned}
x &= y^3 \\
a + b &= c \\
x &= y - 2 \\
a + b &= d + e \\
x^2 + 3 &= y \\
a(x) &= 2x \\
b_i &= 5x \\
10x^2 &= 9x \\
2x^2 + 3x + 2 &= 0 \\
5x - 2 &= 0 \\
d &= \log x \\
y &= \sin x
\end{aligned}
\tag{2}$$

LAMPIRAN

L.6 Format Penulisan Referensi

Penulisan referensi mengikuti aturan standar yang sudah ditentukan. Untuk internasionalisasi DTETI, maka penulisan referensi akan mengikuti standar yang ditetapkan oleh IEEE (*International Electronics and Electrical Engineers*). Aturan penulisan ini bisa diunduh di <http://www.ieee.org/documents/ieeecitationref.pdf>. Gunakan Mendeley sebagai *reference manager* dan *export* data ke format Bibtex untuk digunakan di Latex.

Berikut ini adalah sampel penulisan dalam format IEEE:

L.6.1 Book

Basic Format:

- [1] J. K. Author, "Title of chapter in the book," in Title of His Published Book, xth ed. City of Publisher, Country: Abbrev. of Publisher, year, ch. x, sec. x, pp. xxx-xxx.

Examples:

- [1] B. Klaus and P. Horn, Robot Vision. Cambridge, MA: MIT Press, 1986.
- [2] L. Stein, "Random patterns," in Computers and You, J. S. Brake, Ed. New York: Wiley, 1994, pp. 55-70.
- [3] R. L. Myer, "Parametric oscillators and nonlinear materials," in Nonlinear Optics, vol. 4, P. G. Harper and B. S. Wherret, Eds. San Francisco, CA: Academic, 1977, pp. 47-160.
- [4] M. Abramowitz and I. A. Stegun, Eds., Handbook of Mathematical Functions (Applied Mathematics Series 55). Washington, DC: NBS, 1964, pp. 32-33.
- [5] E. F. Moore, "Gedanken-experiments on sequential machines," in Automata Studies (Ann. of Mathematical Studies, no. 1), C. E. Shannon and J. McCarthy, Eds. Princeton, NJ: Princeton Univ. Press, 1965, pp. 129-153.
- [6] Westinghouse Electric Corporation (Staff of Technology and Science, Aerospace Div.), Integrated Electronic Systems. Englewood Cliffs, NJ: Prentice-Hall, 1970.
- [7] M. Gorkii, "Optimal design," Dokl. Akad. Nauk SSSR, vol. 12, pp. 111-122, 1961 (Transl.: in L. Pontryagin, Ed., The Mathematical Theory of Optimal Processes. New York: Interscience, 1962, ch. 2, sec. 3, pp. 127-135).
- [8] G. O. Young, "Synthetic structure of industrial plastics," in Plastics, vol. 3,

Polymers of Hexadromicon, J. Peters, Ed., 2nd ed. New York: McGraw-Hill, 1964, pp. 15-64.

L.6.2 Handbook

Basic Format:

- [1] Name of Manual/Handbook, x ed., Abbrev. Name of Co., City of Co., Abbrev. State, year, pp. xx-xx.

Examples:

- [1] Transmission Systems for Communications, 3rd ed., Western Electric Co., Winston Salem, NC, 1985, pp. 44-60.
- [2] Motorola Semiconductor Data Manual, Motorola Semiconductor Products Inc., Phoenix, AZ, 1989.
- [3] RCA Receiving Tube Manual, Radio Corp. of America, Electronic Components and Devices, Harrison, NJ, Tech. Ser. RC-23, 1992.

Conference/Prosiding

Basic Format:

- [1] J. K. Author, "Title of paper," in Unabbreviated Name of Conf., City of Conf., Abbrev. State (if given), year, pp.xxx-xxx.

Examples:

- [1] J. K. Author [two authors: J. K. Author and A. N. Writer] [three or more authors: J. K. Author et al.], "Title of Article," in [Title of Conf. Record as], [copyright year] © [IEEE or applicable copyright holder of the Conference Record]. doi: [DOI number]

Sumber Online/Internet

Basic Format:

- [1] J. K. Author. (year, month day). Title (edition) [Type of medium]. Available: [http://www.\(URL\)](http://www.(URL))

Examples:

- [1] J. Jones. (1991, May 10). Networks (2nd ed.) [Online]. Available: <http://www.atm.com>

Skripsi, Tesis dan Disertasi

Basic Format:

- [1] J. K. Author, "Title of thesis," M.S. thesis, Abbrev. Dept., Abbrev. Univ., City of Univ., Abbrev. State, year.

[2] J. K. Author, "Title of dissertation," Ph.D. dissertation, Abbrev. Dept., Abbrev. Univ., City of Univ., Abbrev. State, year.

Examples:

[1] J. O. Williams, "Narrow-band analyzer," Ph.D. dissertation, Dept. Elect. Eng., Harvard Univ., Cambridge, MA, 1993. [2] N. Kawasaki, "Parametric study of thermal and chemical nonequilibrium nozzle flow," M.S. thesis, Dept. Electron. Eng., Osaka Univ., Osaka, Japan, 1993

LAMPIRAN

L.7 Cuplikan kode implementasi I2S

L.7.1 Cuplikan RTL: sinkronisasi register AXI ke domain `audio_clk`

```
1 always @(posedge audio_clk or posedge audio_rst)
2 begin
3     if (audio_rst)
4     begin
5         ctrl_sync_a    <= 32'd0;
6         ctrl_sync_b    <= 32'd0;
7         sample_left_q  <= 32'd0;
8         sample_right_q <= 32'd0;
9     end
10    else
11    begin
12        ctrl_sync_a    <= slv_reg2;    // CONTROL @ 0x08
13        ctrl_sync_b    <= ctrl_sync_a;
14        sample_left_q  <= slv_reg0;    // DATA_LEFT @ 0x00
15        sample_right_q <= slv_reg1;    // DATA_RIGHT @ 0x04
16    end
17 end
```

L.7.2 Cuplikan RTL: generator sinyal I2S Philips

```
1 wire bclk_en_w = fs_mode_sync ? (clock_div_q[0] == 1'b0)
2                               : (clock_div_q[1:0] == 2'd0);
3
4 always @(posedge audio_clk or posedge audio_rst)
5 begin
6     if (audio_rst)
7     begin
8         bit_count_q <= 6'd0;
9         data_q      <= 1'b0;
10        ws_q        <= 1'b0;
11        bclk_q       <= 1'b0;
12    end
13    else if (bclk_en_w)
14    begin
15        if (!enable_sync)
16        begin
17            bit_count_q <= 6'd0;
```



```

18         data_q      <= 1'b0;
19         ws_q        <= 1'b0;
20         bclk_q       <= 1'b0;
21     end
22     else if (bclk_q)
23     begin
24         bclk_q      <= 1'b0;
25         data_q      <= i2s_next_serial_bit(
sample_for_i2s_left ,
26
sample_for_i2s_right ,
27
bit_count_q ,
28
sample_width_sync
);
29         ws_q        <= bit_count_q[5]; // 0=left slot , 1=
right slot
30         bit_count_q <= bit_count_q + 6'd1;
31     end
32     else
33         bclk_q <= 1'b1;
34 end
35 end

```

L.7.3 Cuplikan aplikasi uji Vitis (bare-metal)

```

1 #define I2S_BASE      XPAR_I2S_0_BASEADDR
2 #define I2S_DATA_L    (I2S_BASE + 0x00U)
3 #define I2S_DATA_R    (I2S_BASE + 0x04U)
4 #define I2S_CONTROL   (I2S_BASE + 0x08U)
5
6 #define CTRL_ENABLE    (1U << 0)
7 #define CTRL_MUTE      (1U << 1)
8 #define CTRL_FS_MODE   (1U << 2)
9 #define CTRL_FS_FAMILY (1U << 3)
10 #define CTRL_SAMPLE_WIDTH(w) (((w) & 0x1FU) << 8)
11
12 static inline void i2s_write_sample(u32 left , u32 right) {
13     Xil_Out32(I2S_DATA_L, left);
14     Xil_Out32(I2S_DATA_R, right);
15 }
16
17 int main(void) {

```

```

18     init_platform ();
19
20     // ENABLE=1, MUTE=0, FS_MODE=0, FS_FAMILY=0, SAMPLE_WIDTH=24
21     Xil_Out32(I2S_CONTROL, CTRL_ENABLE | CTRL_SAMPLE_WIDTH(24));
22
23     while (1) {
24         i2s_write_sample(0x00007FFF, 0x00007FFF);
25         usleep(1000000U / 48000U);
26     }
27 }

```

L.7.4 Register map I2S

Tabel L.5. Ringkasan register peripheral I2S

Offset	Register	Keterangan
0x00	DATA_LEFT	Register data kanal kiri (payload ditempatkan pada bit rendah sesuai SAMPLE_WIDTH)
0x04	DATA_RIGHT	Register data kanal kanan (payload ditempatkan pada bit rendah sesuai SAMPLE_WIDTH)
0x08	CONTROL	Bit [0] ENABLE, [1] MUTE, [2] FS_MODE, [3] FS_FAMILY, [12:8] SAMPLE_WIDTH
0x0C	RESERVED	Cadangan pengembangan lanjutan (status/interrupt pada versi berikutnya)